Sección 2: Arquitectura, Componentes y Hooks en React Native

Entregable de Investigación Técnica – Persona 2 (25 %)

2.1. Arquitectura básica de una aplicación React Native

React Native es un framework de desarrollo móvil multiplataforma creado por Meta Platforms que permite construir aplicaciones nativas para Android e iOS utilizando JavaScript (o TypeScript) y la biblioteca React. A diferencia de los enfoques híbridos basados en WebViews (como Apache Cordova o Ionic en sus primeras versiones), React Native traduce la jerarquía de componentes declarada en React directamente en vistas nativas del sistema operativo anfitrión (Meta Platforms, 2025a).

2.1.1. Hilos principales de ejecución

El entorno de ejecución opera sobre tres hilos que trabajan de forma coordinada:

1. **Hilo de JavaScript (JS Thread):** ejecuta la lógica de la aplicación con el motor Hermes (por defecto), procesa el árbol de React, gestiona los estados (Hooks) y despacha las solicitudes de red.
2. **Hilo principal o nativo (Main / UI Thread):** pinta la interfaz en pantalla y captura los eventos táctiles del usuario.
3. **Hilo de sombra (Shadow Thread):** utiliza el motor de diseño Yoga (C++) para calcular posiciones y dimensiones de los elementos definidos con Flexbox antes de enviarlas al hilo nativo.

2.1.2. Evolución arquitectónica: del Bridge a la Nueva Arquitectura

En su diseño clásico, React Native comunicaba el hilo de JavaScript y el hilo nativo mediante un puente asíncrono (Bridge) que serializaba mensajes en JSON. Este proceso generaba cuellos de botella al mover grandes volúmenes de datos o al desplazar listas extensas. Las versiones modernas implementan la **Nueva Arquitectura** (Meta Platforms, 2025b), compuesta por:

- **JSI (JavaScript Interface):** capa en C++ que reemplaza al Bridge; permite que JavaScript mantenga referencias directas a objetos nativos e invoque métodos sin serializar JSON.
- **Fabric:** nuevo sistema de renderizado compartido en C++, con soporte para renderizado concurrente y priorización de eventos urgentes.
- **TurboModules:** módulos nativos (cámara, sensores, red) que se cargan bajo demanda (lazy loading), reduciendo el tiempo de arranque.
- **Codegen:** genera código nativo tipado a partir de definiciones en JavaScript, garantizando seguridad de tipos entre capas.

2.2. Estructura y organización de componentes

En React Native la interfaz se construye con **componentes funcionales**: funciones independientes, reutilizables y declarativas que reciben propiedades inmutables (props), pueden administrar estado local y retornan una descripción visual en JSX (Boduch & Derks, 2020).

La interfaz se organiza como un árbol jerárquico con flujo de datos unidireccional (de arriba hacia abajo):

- **Componente raíz (App.js):** punto de entrada que monta la pantalla principal.
- **Pantallas o contenedores (src/screens/):** coordinan el estado general y distribuyen la información. Ejemplo: HomeScreen.js.
- **Componentes reutilizables (src/components/):** reciben datos mediante props y notifican acciones con funciones callback. Ejemplos: ItemCard.js y CategoryFilter.js.
- **Capa de datos y estilos (src/data/ y src/styles/):** datos de prueba (mock data), funciones normalizadoras y constantes globales de diseño.

Estructura de carpetas utilizada en el proyecto:

```
persona2-interfaz/
|-- App.js                    # Punto de entrada
|-- src/
    |-- components/           # Componentes reutilizables
    |   |-- ItemCard.js       # Tarjeta de cada registro
    |   |-- CategoryFilter.js # Barra horizontal de filtros
    |-- screens/              # Pantallas principales
    |   |-- HomeScreen.js     # Listado principal con FlatList
    |-- data/                 # Datos de prueba y adaptadores
    |   |-- mockData.js
    |-- services/             # Consumo de APIs REST (Persona 3)
    |-- styles/               # Constantes y estilos compartidos
        |-- theme.js
```

2.3. Componentes principales: View, Text, FlatList, Image, ScrollView y StyleSheet

React Native provee componentes nucleares (Core Components) que se traducen a controles nativos en cada plataforma (Meta Platforms, 2025a):

- **View:** contenedor básico para estructurar la interfaz (equivale a android.view.View en Android y UIView en iOS). Usa Flexbox con dirección vertical por defecto (flexDirection: 'column').
- **Text:** único componente válido para mostrar texto. Texto suelto fuera de un Text produce error en tiempo de ejecución. Admite numberOfLines para truncar contenido.
- **Image:** muestra imágenes locales (require) o remotas (source={{ uri: '...' }}). Las remotas requieren ancho y alto explícitos; resizeMode (cover, contain, stretch) controla el ajuste.
- **ScrollView:** contenedor desplazable que renderiza todos sus hijos en memoria al mismo tiempo. Es ideal para vistas cortas o barras horizontales de categorías (horizontal={true}).
- **FlatList:** componente de alto rendimiento para listas mediante virtualización: solo mantiene en memoria los elementos visibles. Propiedades esenciales: data (arreglo), renderItem (función que dibuja cada elemento), keyExtractor (identificador único por registro), ListHeaderComponent y ListEmptyComponent.

- **StyleSheet:** API de estilos (StyleSheet.create({})) con sintaxis camelCase similar a CSS. Las dimensiones son independientes de la densidad de píxeles (dp en Android, pt en iOS).

Comparativa técnica entre ScrollView y FlatList:

Criterio técnico	ScrollView	FlatList
Estrategia de renderizado	Renderiza todos los hijos al montarse.	Virtualización: solo renderiza los elementos visibles.
Consumo de memoria	Crece con la cantidad de elementos.	Se mantiene controlado aunque haya muchos registros.
Caso de uso en el proyecto	Barra de filtros por categoría (CategoryFilter.js).	Listado principal de tarjetas (HomeScreen.js).

Ejemplo integrador de los componentes principales:

```
import React from 'react';
import { View, Text, Image, FlatList, ScrollView, StyleSheet } from 'react-native';

const DATOS = [
  { id: '1', titulo: 'React Native', imagen: 'https://picsum.photos/200' },
  { id: '2', titulo: 'Expo SDK', imagen: 'https://picsum.photos/201' },
];

export default function EjemploComponentes() {
  return (
    <View style={styles.contenedor}>
      <ScrollView horizontal showsHorizontalScrollIndicator={false}>
        <Text style={styles.etiqueta}>Todos</Text>
        <Text style={styles.etiqueta}>Destacados</Text>
      </ScrollView>

      <FlatList
        data={DATOS}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <View style={styles.tarjeta}>
            <Image source={{ uri: item.imagen }} style={styles.imagen} />
            <Text style={styles.titulo}>{item.titulo}</Text>
          </View>
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  contenedor: { flex: 1, padding: 16, backgroundColor: '#F8FAFC' },
  etiqueta: { marginRight: 10, padding: 8, backgroundColor: '#DBEAFE', borderRadius: 8 },
  tarjeta: { flexDirection: 'row', alignItems: 'center', marginVertical: 8, padding: 12, backgroundColo
  imagen: { width: 50, height: 50, borderRadius: 25, marginRight: 12 },
  titulo: { fontSize: 16, fontWeight: 'bold', color: '#0F172A' },
});
```

2.4. Funcionamiento de useState y useEffect, con ejemplos

Los Hooks son funciones de React que permiten usar estado y ciclo de vida en componentes funcionales (Meta Platforms, 2025c). Deben invocarse solo en el nivel superior del componente y nunca dentro de condicionales o bucles.

2.4.1. Gestión de estado local con useState

useState declara una variable de estado que se conserva entre renderizados. Devuelve el valor actual y una función para actualizarlo; al actualizarla, React vuelve a renderizar el componente. Sintaxis: const [estado, setEstado] = useState(valorInicial);

```
import React, { useState } from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

export default function EjemploUseState() {
  const [favorito, setFavorito] = useState(false);

  return (
    <View style={styles.contenedor}>
      <Text style={styles.estado}>
        Estado: {favorito ? 'Guardado en Favoritos' : 'Sin guardar'}
      </Text>
      <TouchableOpacity style={styles.boton} onPress={() => setFavorito(!favorito)}>
        <Text style={styles.textoBoton}>Cambiar estado</Text>
      </TouchableOpacity>
    </View>
  );
}

const styles = StyleSheet.create({
  contenedor: { padding: 16, alignItems: 'center' },
  estado: { fontSize: 15, marginBottom: 10 },
  boton: { backgroundColor: '#2563EB', paddingVertical: 8, paddingHorizontal: 16, borderRadius: 8 },
  textoBoton: { color: '#FFFFFF', fontWeight: 'bold' },
});
```

2.4.2. Efectos secundarios y ciclo de vida con useEffect

useEffect ejecuta efectos secundarios después del renderizado: cargar datos, realizar peticiones HTTP a una API REST, configurar temporizadores o suscribirse a eventos. Sintaxis:

```
useEffect(() => {
  // 1. Efecto secundario
  return () => {
    // 2. Limpieza (al desmontar o antes de volver a ejecutar)
  };
}, [dependencias]);
```

El comportamiento depende del arreglo de dependencias:

- **[] (vacío):** se ejecuta una sola vez al montar el componente. Es el patrón habitual para la carga inicial de datos.
- **[variable]:** se ejecuta al montar y cada vez que la variable cambia.
- **Sin arreglo:** se ejecuta después de cada renderizado; debe evitarse si modifica el estado, porque puede generar un bucle infinito.

```
import React, { useState, useEffect } from 'react';
import { View, Text, FlatList, ActivityIndicator } from 'react-native';

const DATOS_PRUEBA = [
  { id: '1', nombre: 'Elemento Alfa' },
  { id: '2', nombre: 'Elemento Beta' },
];

export default function ListaConCicloDeVida() {
  const [datos, setDatos] = useState([]);
  const [cargando, setCargando] = useState(true);

  useEffect(() => {
    const temporizador = setTimeout(() => {
      setDatos(DATOS_PRUEBA);
```

```
      setCargando(false);
    }, 600);
    return () => clearTimeout(temporizador);
  }, []);

  if (cargando) {
    return <ActivityIndicator size="large" color="#2563EB" />;
  }

  return (
    <View style={{ flex: 1, padding: 16 }}>
      <FlatList
        data={datos}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => <Text>{item.nombre}</Text>}
      />
    </View>
  );
}
```

2.5. Referencias bibliográficas (APA 7.ª edición)

Boduch, A., & Derks, R. (2020). React and React Native: A complete hands-on guide to modern web and mobile development with React.js (3rd ed.). Packt Publishing.

Eisenman, B. (2017). Learning React Native: Building native mobile apps with JavaScript (2nd ed.). O'Reilly Media.

Expo. (2025). Expo documentation: Develop, review, and deploy universal native apps. https://docs.expo.dev/

Meta Platforms. (2025a). React Native documentation: Core components and APIs. https://reactnative.dev/docs/components-and-apis

Meta Platforms. (2025b). React Native documentation: About the New Architecture. https://reactnative.dev/architecture/landing-page

Meta Platforms. (2025c). React documentation: Built-in React Hooks. https://react.dev/reference/react/hooks